

SOLUCIONARIO - TUTORÍA LC2

MiniMarket Normalizado | Semana 4

Indicaciones de uso

- Este documento es una guía resuelta para tutoría; puede compartirse después de que los estudiantes intenten los ejercicios.
- El código usa la misma base de datos minimarket_normalizacion.db.
- `display(df)` solo muestra el resultado como tabla en Google Colab; no modifica la base de datos.
- `pd.read_sql_query(...)` ejecuta la consulta SQL y devuelve un DataFrame de pandas.

Ejercicio 1. Inventario inicial de tablas del minimarket

Pregunta guía: ¿Qué tablas forman el modelo normalizado? ¿Cuál parece ser la tabla central del negocio y por qué?

Código

```
# Ver las tablas disponibles
df_tablas = pd.read_sql_query(
    '''
    SELECT name AS tabla
    FROM sqlite_schema
    WHERE type = 'table'
    AND name NOT LIKE 'sqlite_%'
    ORDER BY name
    ''',
    conn
)
display(df_tablas)
```

Resultado esperado

tabla
categorias
clientes
locales
productos
ventas
ventas_original

Respuesta guía

El modelo contiene las tablas categorías, clientes, locales, productos, ventas y ventas_original. La tabla central del modelo normalizado es ventas, porque registra cada operación del negocio y conecta clientes, productos y locales mediante claves foráneas.

Ejercicio 2. Catálogo de productos con su categoría

Pregunta guía: ¿Por qué no basta con consultar solo productos si se desea ver el nombre de la categoría?

Código

```
# Productos con el nombre de su categoría
df_catalogo = pd.read_sql_query(
    '''
    SELECT
        p.id_producto,
        p.nombre_producto,
        cat.nombre_categoria,
        p.precio_unitario
    FROM productos p
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    ORDER BY cat.nombre_categoria, p.nombre_producto
    ''',
    conn
)
display(df_catalogo)
```

Resultado esperado

id_producto	nombre_producto	nombre_categoria	precio_unitario
2	Aceite Primor 1L	Abarrotes	12.80
1	Arroz Costeno 5kg	Abarrotes	28.50
3	Fideo Don Vittorio 500g	Abarrotes	4.20
5	Agua San Luis 2.5L	Bebidas	3.80
4	Gaseosa Inca Kola 3L	Bebidas	10.50
8	Jabon Protex	Cuidado personal	4.80
6	Detergente Bolivar 1kg	Limpieza	16.50
7	Lejia Clorox 1L	Limpieza	5.90

Respuesta guía

No basta con productos porque allí solo aparece id_categoria. El INNER JOIN permite traer el nombre de la categoría desde la tabla categorias. Esta separación evita repetir el nombre de la categoría en cada producto y mantiene el diseño normalizado.

Ejercicio 3. Boleta resumida de ventas recientes

Pregunta guía: ¿Qué información aporta cada una de las tablas unidas?

Código

```
# Boleta resumida desde el 5 de abril de 2026
df_boleta = pd.read_sql_query(
    '''
    SELECT
        v.id_venta,
        v.fecha,
```

```

        c.nombre_cliente,
        c.districto_cliente,
        p.nombre_producto,
        l.nombre_local,
        v.cantidad,
        p.precio_unitario,
        ROUND(v.cantidad * p.precio_unitario, 2) AS total_venta
FROM ventas v
INNER JOIN clientes c ON v.id_cliente = c.id_cliente
INNER JOIN productos p ON v.id_producto = p.id_producto
INNER JOIN locales l ON v.id_local = l.id_local
WHERE v.fecha >= '2026-04-05'
ORDER BY v.fecha, v.id_venta
'''
conn
)
display(df_boleta)

```

Resultado esperado

id_venta	fecha	nombre_cliente	districto_cliente	nombre_producto	nombre_local	cantidad	precio_unitario	total_venta
9	2026-04-05	Jorge Medina	Los Olivos	Jabon Protex	MiniMarket Central	3	4.80	14.40
10	2026-04-05	Maria Perez	Independencia	Gaseosa Inca Kola 3L	MiniMarket Plaza	2	10.50	21.00
11	2026-04-06	Carlos Rios	San Martin	Detergente Bolivar 1kg	MiniMarket Norte	2	16.50	33.00
12	2026-04-06	Rosa Salazar	Puente Piedra	Aceite Primor 1L	MiniMarket Central	1	12.80	12.80

Respuesta guía

ventas aporta fecha, cantidad e identificadores; clientes aporta nombre y distrito del cliente; productos aporta nombre y precio; locales aporta el local donde ocurrió la venta. Sin JOIN solo se verían códigos como id_cliente, id_producto e id_local.

Ejercicio 4. Distritos de clientes con menor actividad

Pregunta guía: ¿Qué distrito debería priorizar marketing y por qué se usa LEFT JOIN?

Código

```

# Actividad comercial por distrito del cliente
df_distritos = pd.read_sql_query(
    '''
    SELECT
        c.districto_cliente,
        COUNT(DISTINCT c.id_cliente) AS clientes_registrados,
        COUNT(v.id_venta) AS compras_registradas,
        ROUND(
            SUM(COALESCE(v.cantidad * p.precio_unitario, 0)),
            2
        ) AS ingresos_generados
    FROM clientes c
    LEFT JOIN ventas v ON c.id_cliente = v.id_cliente
    '''
)
display(df_distritos)

```

```

LEFT JOIN productos p ON v.id_producto = p.id_producto
GROUP BY c.districto_cliente
ORDER BY ingresos_generados ASC
'''
conn
)
display(df_districtos)

```

Resultado esperado

districto_cliente	clientes_registrados	compras_registradas	ingresos_generados
Puente Piedra	1	2	24.60
Independencia	1	2	37.50
San Martin	1	2	49.80
Comas	2	2	81.00
Los Olivos	2	4	132.60

Respuesta guía

Marketing debería priorizar Puente Piedra, porque registra el menor ingreso generado: 24.60. Se usa LEFT JOIN porque la consulta parte de clientes y busca conservarlos aunque tengan pocas o ninguna compra. Con INNER JOIN se podrían perder clientes sin ventas.

Ejercicio 5. Categorías más fuertes por local

Pregunta guía: ¿Qué categoría conviene reforzar en cada local?

Código

```

# Ingresos por local y categoría
df_local_categoria = pd.read_sql_query(
    '''
    SELECT
        l.nombre_local,
        cat.nombre_categoria,
        COUNT(v.id_venta) AS cantidad_ventas,
        SUM(v.cantidad) AS unidades_vendidas,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
    FROM ventas v
    INNER JOIN locales l ON v.id_local = l.id_local
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    GROUP BY
        l.id_local, l.nombre_local,
        cat.id_categoria, cat.nombre_categoria
    ORDER BY l.nombre_local, ingresos_totales DESC
    '''
    conn
)
display(df_local_categoria)

```

Resultado esperado

nombre_local	nombre_categoria	cantidad_ventas	unidades_vendidas	ingresos_totales
--------------	------------------	-----------------	-------------------	------------------

MiniMarket Central	Abarrotes	3	6	108.20
MiniMarket Central	Bebidas	1	6	22.80
MiniMarket Central	Cuidado personal	1	3	14.40
MiniMarket Norte	Bebidas	1	5	52.50
MiniMarket Norte	Limpieza	2	4	44.80
MiniMarket Norte	Abarrotes	1	1	28.50
MiniMarket Plaza	Bebidas	1	2	21.00
MiniMarket Plaza	Abarrotes	1	4	16.80

Nota: se muestran 8 de 9 filas.

Respuesta guía

En MiniMarket Central conviene reforzar Abarrotes, con 108.20 de ingresos. En MiniMarket Norte conviene reforzar Bebidas, con 52.50. En MiniMarket Plaza conviene reforzar Bebidas, con 21.00. GROUP BY resume muchas ventas individuales en información útil para abastecimiento.

Ejercicio 6. Categorías con ticket promedio relevante

Pregunta guía: ¿Por qué se usa HAVING y qué categoría parece más atractiva?

Código

```
# Categorías con al menos 3 ventas y ticket promedio mayor a 25 soles
df_ticket_categoria = pd.read_sql_query(
    '''
    SELECT
        cat.nombre_categoria,
        COUNT(v.id_venta) AS cantidad_ventas,
        ROUND(AVG(v.cantidad * p.precio_unitario), 2) AS ticket_promedio,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS ingresos_totales
    FROM ventas v
    INNER JOIN productos p ON v.id_producto = p.id_producto
    INNER JOIN categorias cat ON p.id_categoria = cat.id_categoria
    GROUP BY cat.id_categoria, cat.nombre_categoria
    HAVING cantidad_ventas >= 3
        AND ticket_promedio > 25
    ORDER BY ticket_promedio DESC
    ''',
    conn
)
display(df_ticket_categoria)
```

Resultado esperado

nombre_categoria	cantidad_ventas	ticket_promedio	ingresos_totales
Bebidas	3	32.10	96.30
Abarrotes	5	30.70	153.50

Respuesta guía

Se usa HAVING porque cantidad_ventas y ticket_promedio son resultados del agrupamiento. WHERE filtra filas antes de agrupar, pero HAVING filtra grupos ya calculados. Por ticket promedio, Bebidas queda primero con 32.10; Abarrotes genera más ingresos totales con 153.50.

Ejercicio 7. Días con mayor movimiento de ventas

Pregunta guía: ¿Qué días tuvieron más movimiento y por qué no se usa JOIN?

Código

```
# Días con dos o más ventas registradas
df_dias_movimiento = pd.read_sql_query(
    '''
    SELECT
        fecha,
        COUNT(id_venta) AS cantidad_ventas,
        SUM(cantidad) AS unidades_vendidas
    FROM ventas
    GROUP BY fecha
    HAVING cantidad_ventas >= 2
    ORDER BY fecha
    ''',
    conn
)
display(df_dias_movimiento)
```

Resultado esperado

fecha	cantidad_ventas	unidades_vendidas
2026-04-01	2	7
2026-04-02	2	4
2026-04-03	2	7
2026-04-04	2	6
2026-04-05	2	5
2026-04-06	2	3

Respuesta guía

Todos los días mostrados tienen 2 ventas registradas. Los días con más unidades vendidas son 2026-04-01 y 2026-04-03, ambos con 7 unidades. No se usa JOIN porque la pregunta se responde solo con datos de la tabla ventas: fecha, id_venta y cantidad.

Ejercicio 8. Redundancia de productos en la tabla original

Pregunta guía: ¿Qué producto se repite más y qué problema puede generar?

Código

```
# Productos repetidos en la tabla original desnormalizada
```

```
df_redundancia_producto = pd.read_sql_query(
    '''
    SELECT
        producto,
        categoria,
        COUNT(*) AS veces_repetido,
        ROUND(SUM(total_venta), 2) AS ingresos_en_tabla_original
    FROM ventas_original
    GROUP BY producto, categoria
    HAVING veces_repetido >= 2
    ORDER BY veces_repetido DESC, ingresos_en_tabla_original DESC
    ''',
    conn
)
display(df_redundancia_producto)
```

Resultado esperado

producto	categoria	veces_repetido	ingresos_en_tabla_original
Arroz Costeno 5kg	Abarrotes	2	85.50
Gaseosa Inca Kola 3L	Bebidas	2	73.50
Aceite Primor 1L	Abarrotes	2	51.20
Detergente Bolivar 1kg	Limpieza	2	49.50

Respuesta guía

Los productos repetidos aparecen 2 veces. Entre ellos, Arroz Costeño 5kg concentra el mayor ingreso en la tabla original, con 85.50. Si cambia su nombre o precio, habría que corregir varias filas, lo que puede causar inconsistencias si alguna fila queda sin actualizar.

Ejercicio 9. Impacto de cambiar el nombre de una categoría

Pregunta guía: ¿En qué modelo sería más fácil corregir el nombre de una categoría?

Código

```
# Cuántas filas de ventas_original contienen cada categoría
df_categoria_original = pd.read_sql_query(
    '''
    SELECT
        categoria,
        COUNT(*) AS filas_afectadas_si_cambia_nombre
    FROM ventas_original
    GROUP BY categoria
    ORDER BY filas_afectadas_si_cambia_nombre DESC
    ''',
    conn
)
display(df_categoria_original)

# Productos asociados a cada categoría en el modelo normalizado
df_categoria_normalizada = pd.read_sql_query(
    '''
    SELECT
```

```

        cat.nombre_categoria,
        COUNT(p.id_producto) AS productos_asociados
    FROM categorias cat
    LEFT JOIN productos p ON cat.id_categoria = p.id_categoria
    GROUP BY cat.id_categoria, cat.nombre_categoria
    ORDER BY productos_asociados DESC
    '''
    conn
)
display(df_categoria_normalizada)

```

Resultado esperado: tabla original

categoria	filas_afectadas_si_cambia_nombre
Abarrotes	5
Limpieza	3
Bebidas	3
Cuidado personal	1

Resultado esperado: modelo normalizado

nombre_categoria	productos_asociados
Abarrotes	3
Bebidas	2
Limpieza	2
Cuidado personal	1

Respuesta guía

Sería más fácil corregir el nombre en el modelo normalizado, porque el nombre de la categoría se guarda una sola vez en categorías. En ventas_original, Abarrotes aparece en 5 filas, Limpieza en 3, Bebidas en 3 y Cuidado personal en 1. La 3FN ayuda a evitar repetir datos que dependen de otra entidad.

Ejercicio 10. Clientes de mayor valor para fidelización

Pregunta guía: ¿Qué clientes califican para fidelización y por qué se usa HAVING?

Código

```

# Clientes con monto acumulado mayor a 80 soles
df_clientes_valor = pd.read_sql_query(
    '''
    SELECT
        c.nombre_cliente,
        c.distrito_cliente,
        COUNT(v.id_venta) AS cantidad_compras,
        SUM(v.cantidad) AS unidades_compradas,
        ROUND(SUM(v.cantidad * p.precio_unitario), 2) AS monto_total
    FROM ventas v
    INNER JOIN clientes c ON v.id_cliente = c.id_cliente
    INNER JOIN productos p ON v.id_producto = p.id_producto
    '''
)

```

```
GROUP BY c.id_cliente, c.nombre_cliente, c.districto_cliente
HAVING monto_total > 80
ORDER BY monto_total DESC
'''
conn
)
display(df_clientes_valor)
```

Resultado esperado

nombre_cliente	districto_cliente	cantidad_compras	unidades_compradas	monto_total
Ana Torres	Los Olivos	3	11	118.20
Luis Ramos	Comas	2	6	81.00

Respuesta guía

Califican Ana Torres, con 118.20, y Luis Ramos, con 81.00. Se usa HAVING porque monto_total es un resultado agregado calculado después del GROUP BY; no es una columna directa de ventas.